

Swinburne University of Technology
School of Science, Computing, and Emerging Technologies

SWE40006 - Software Deployment and Evolution
Deployment Portfolio Task 1

Student name: Ngoc Van Nhi Do
Student ID: 105551765
Submission date: 17/08/2026

Deployment Portfolio Task 1

1 Declared Target Level

All four subtasks 1.1, 1.2, 1.3, and 1.4 were attempted. The technologies used include Visual Studio 2022 (v.17.14.37), WiX v7, and the HeatWave extension installed on the IDE.

2 Execution Evidence

Table 1 below contain quick links to source code files created for all deployment tasks, as well as other relevant links as proof of successful task execution.

Table 1: Links to source code files, releases, and PRs

Source	URL
Task 1.1	https://github.com/nhydnv/SWE40006-Software-Deployment-and-Evolution/tree/main/deployment_task_1/subtask_1
Task 1.2	https://github.com/nhydnv/SWE40006-Software-Deployment-and-Evolution/tree/main/deployment_task_1/subtask_2
Task 1.3	https://github.com/nhydnv/SWE40006-Software-Deployment-and-Evolution/tree/main/deployment_task_1/subtask_3
Task 1.4 Github Release	https://github.com/nhydnv/Subtask-4/releases/tag/1.0.0
Pull request to microsoft/winget-pkgs	https://github.com/microsoft/winget-pkgs/pull/418748

2.1 Task 1.1

I built and packaged a sample console application into an `.msi` installer using WiX 7.0.0 via the Command-line Interface (CLI).

The first step is creating a new C# project, adding the provided code, and building it in Release mode.

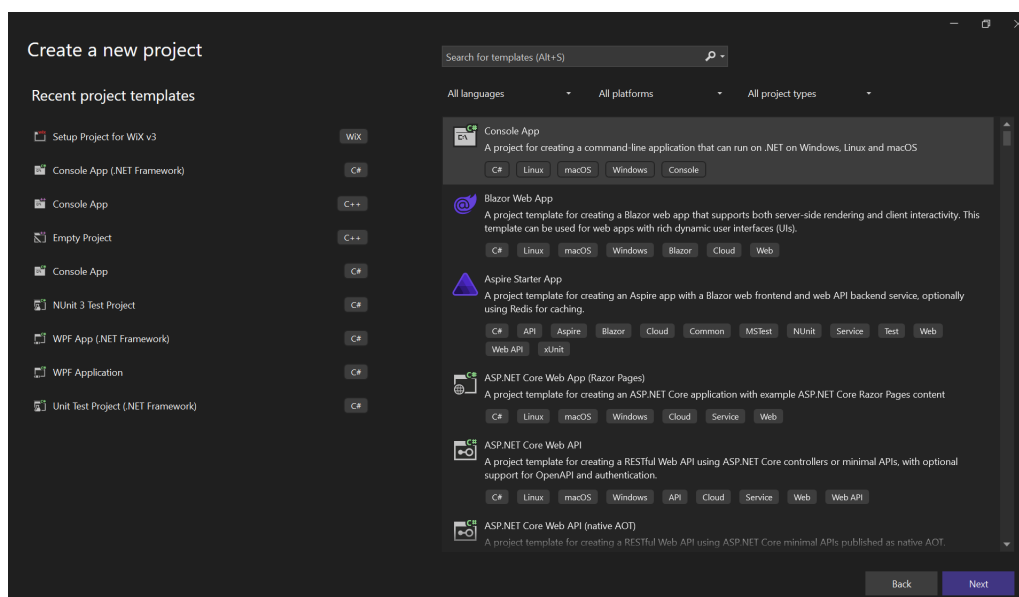


Figure 1: Creating a new C# console application.

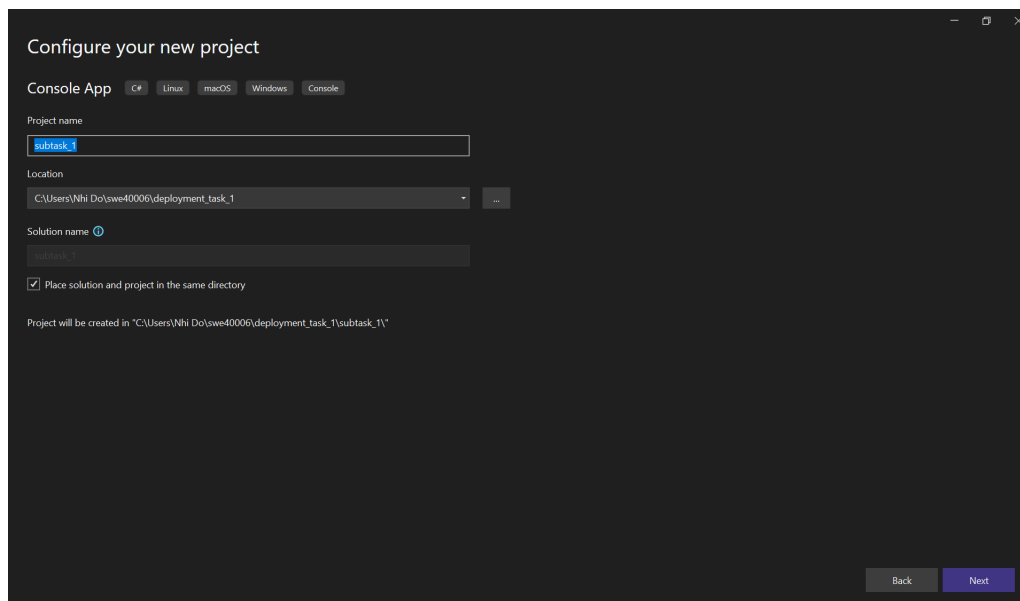


Figure 2: Naming the application.

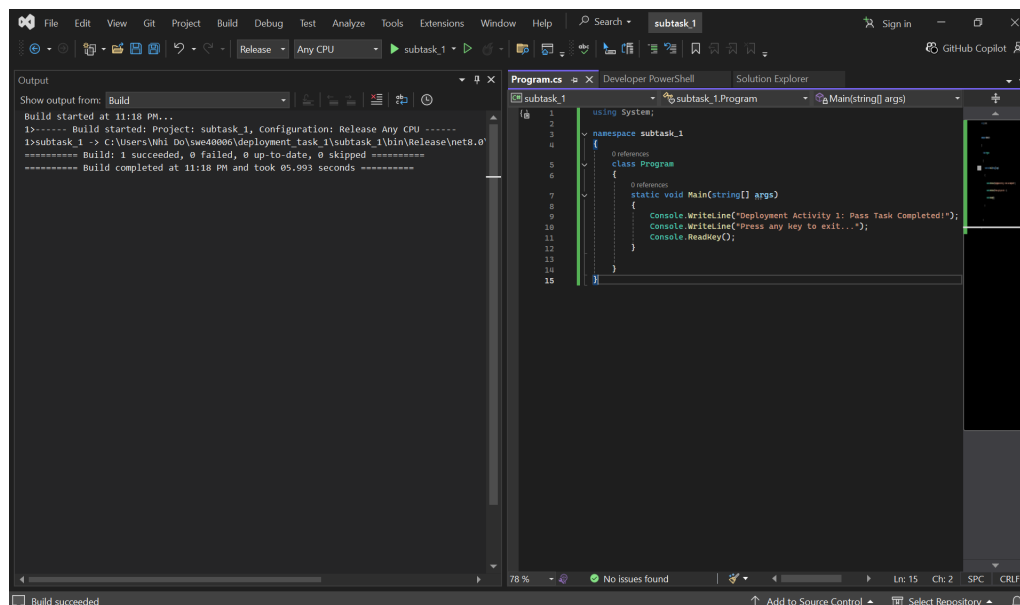
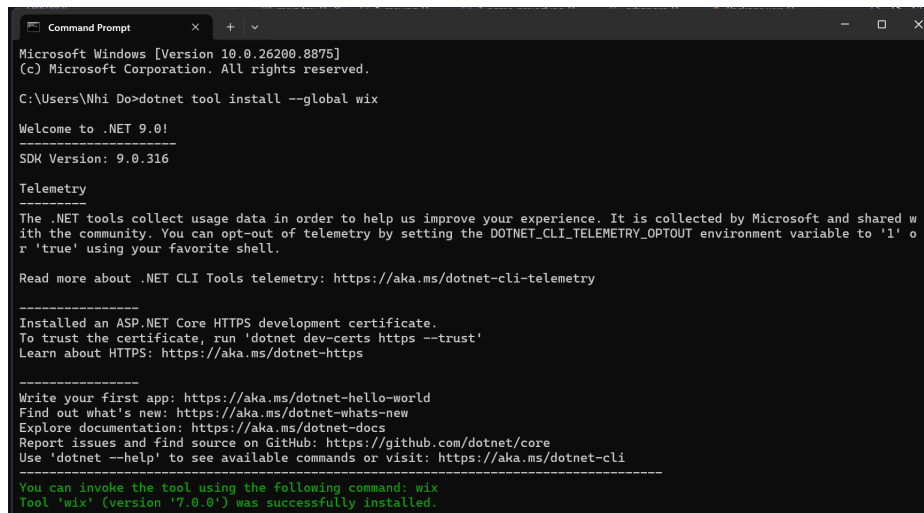


Figure 3: Building the program in Release mode.

Once the program has been built, its executable is created the project's **bin/Release** folder. The WiX toolset is installed using the CLI.



```

Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nhi Do>dotnet tool install --global wix

Welcome to .NET 9.0!
-----
SDK Version: 9.0.316

Telemetry
-----
The .NET tools collect usage data in order to help us improve your experience. It is collected by Microsoft and shared w
ith the community. You can opt-out of telemetry by setting the DOTNET_CLI_TELEMETRY_OPTOUT environment variable to '1' o
r 'true' using your favorite shell.

Read more about .NET CLI Tools telemetry: https://aka.ms/dotnet-cli-telemetry

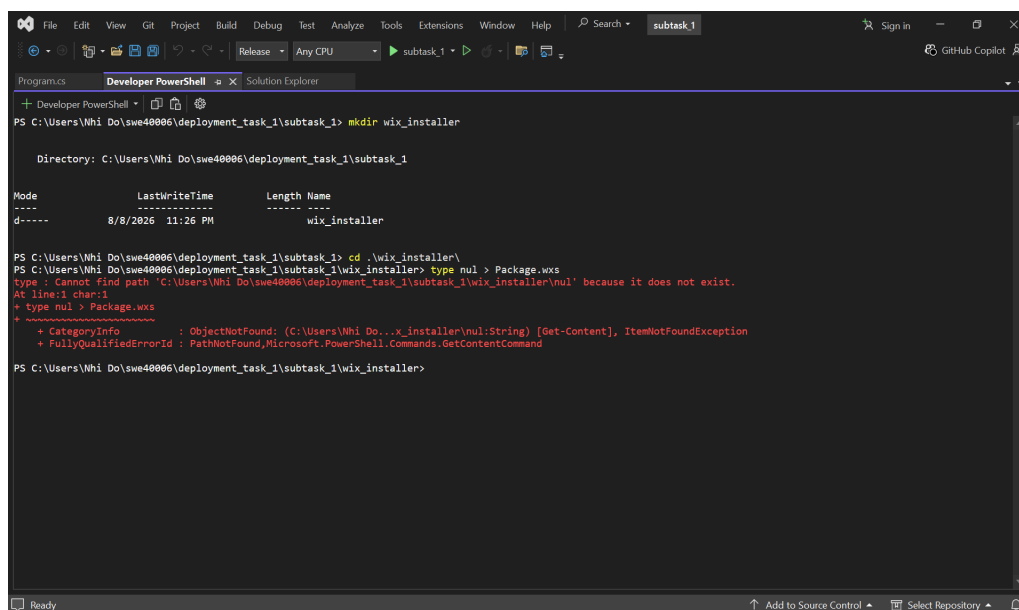
-----
Installed an ASP.NET Core HTTPS development certificate.
To trust the certificate, run 'dotnet dev-certs https --trust'
Learn about HTTPS: https://aka.ms/dotnet-https

-----
Write your first app: https://aka.ms/dotnet-hello-world
Find out what's new: https://aka.ms/dotnet-whats-new
Explore documentation: https://aka.ms/dotnet-docs
Report issues and find source on GitHub: https://github.com/dotnet/core
Use 'dotnet --help' to see available commands or visit: https://aka.ms/dotnet-cli
-----
You can invoke the tool using the following command: wix
Tool 'wix' (version '7.0.0') was successfully installed.

```

Figure 4: Installing WiX.

The WiX project and source file are manually created. The `.wxs` file is then configured to contain instructions to build the installer. The file specifies the application name, manufacturer, version, and upgrade code, as well as the directory in which the application will be installed. It also defines the application files that should be included in the installer and groups them into one feature.



```

PS C:\Users\Nhi Do\swe40006\deployment_task_1\subtask_1> mkdir wix_installer

Directory: C:\Users\Nhi Do\swe40006\deployment_task_1\subtask_1

Mode                LastWriteTime         Length Name
----                -
d-----          8/8/2026 11:26 PM             wix_installer

PS C:\Users\Nhi Do\swe40006\deployment_task_1\subtask_1> cd .\wix_installer\
PS C:\Users\Nhi Do\swe40006\deployment_task_1\subtask_1\wix_installer> type nul > Package.wxs
type : Cannot find path 'C:\Users\Nhi Do\swe40006\deployment_task_1\subtask_1\wix_installer\nul' because it does not exist.
At line:1 char:1
+ type nul > Package.wxs
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\Nhi Do...x_installer\nul:String) [Get-Content], ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.GetContentCommand

PS C:\Users\Nhi Do\swe40006\deployment_task_1\subtask_1\wix_installer>

```

Figure 5: Creating the WiX installer project and source file.

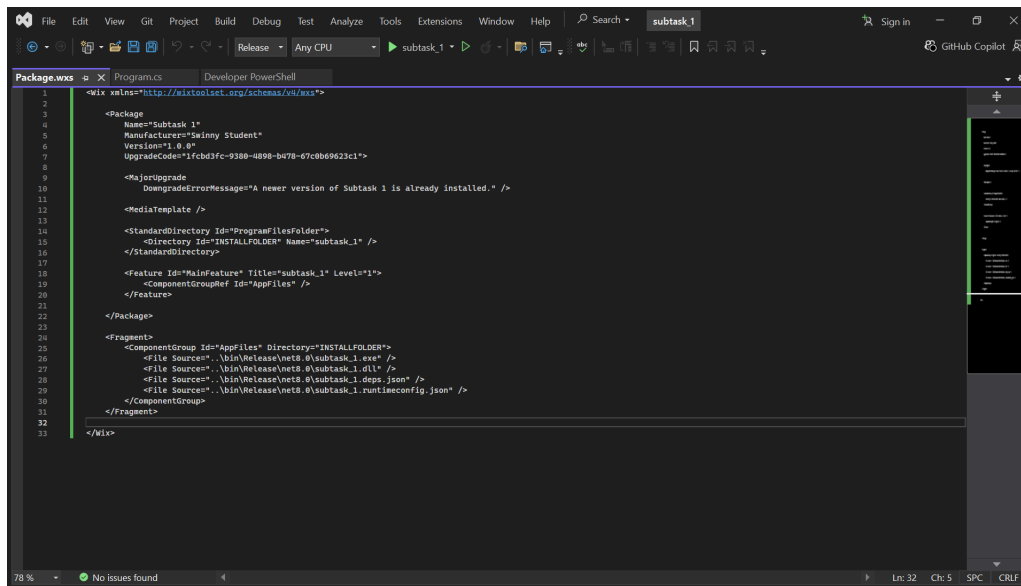


Figure 6: Configuring the Package.wxs file to build the installer.

The installer is then built using the command below. The resulting MSI file is created inside the installer directory.

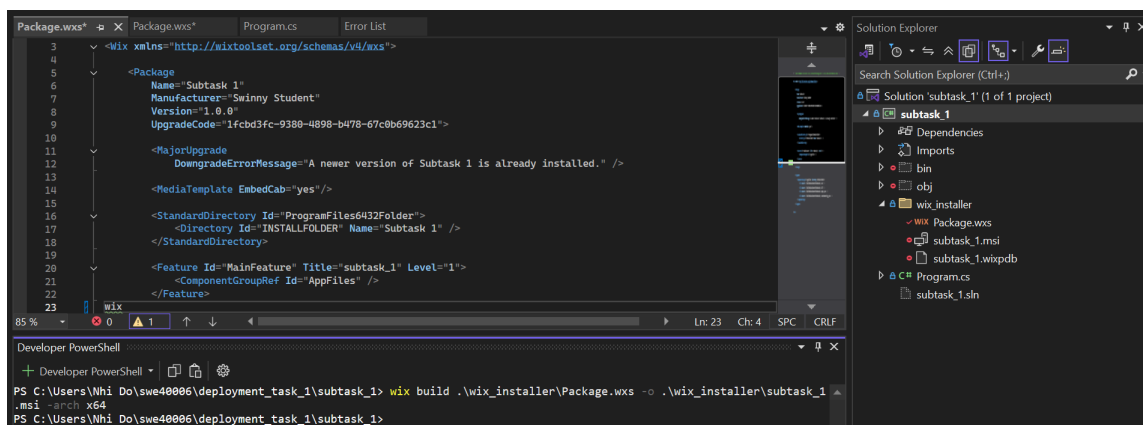


Figure 7: Building the installer from the WiX source file.

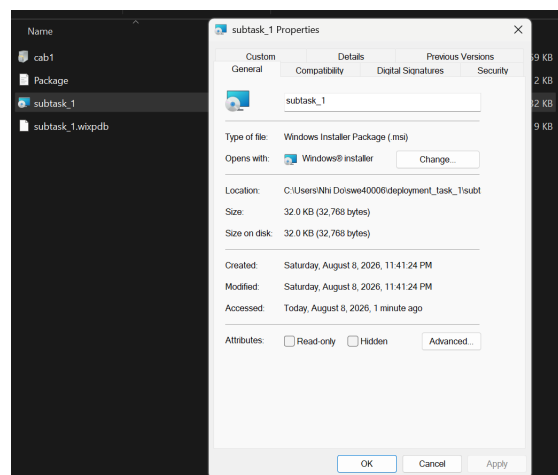


Figure 8: The MSI file is created.

Running the installer **subtask_1.msi** installs the application. The screenshot below shows that the app is successfully installed in the Program Files folder and runs as expected. In the first few

attempts, the executable had been installed in the Program Files (x86) folder instead, which was not ideal considering that it was a 64-bit application. This was resolved by adding the `-arch x64` flag to the `wix build` command to specify the target platform.

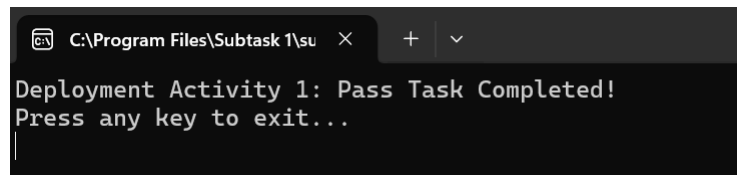


Figure 9: Running the console application after it has been installed.

The application Subtask 1 is finally uninstalled.

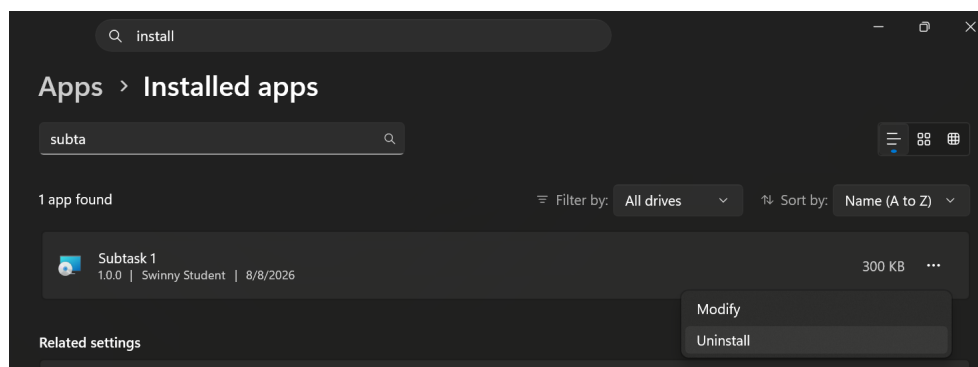


Figure 10: Uninstalling the application.

2.2 Task 1.2

For task 1.2, I built and packaged a C++ console grading application also using the WiX toolset.

First, I created the program and built it in Release mode. The resulting executable code is created in the x64 directory.

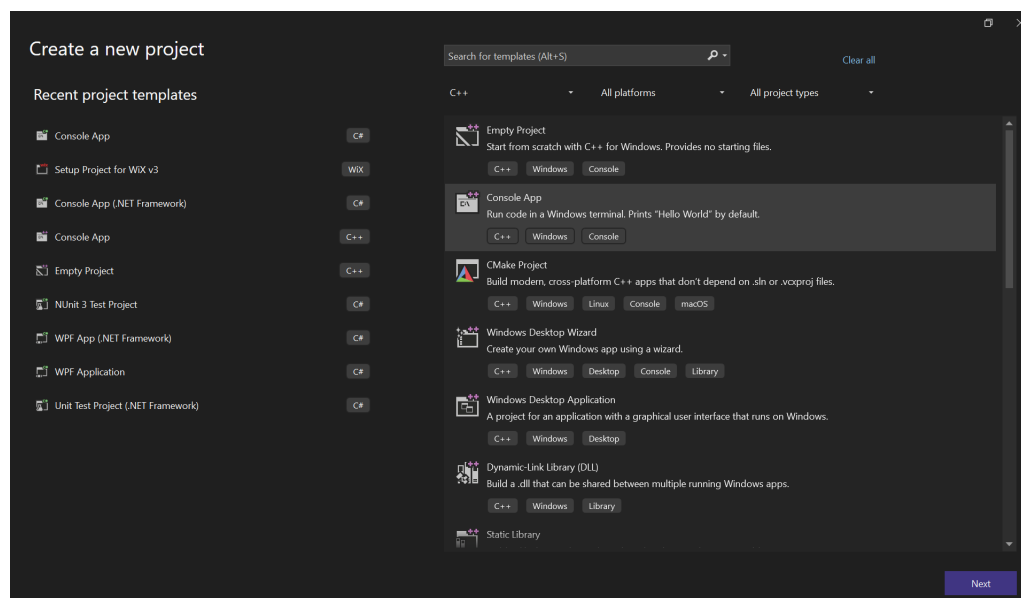


Figure 11: Creating a new C++ console application.

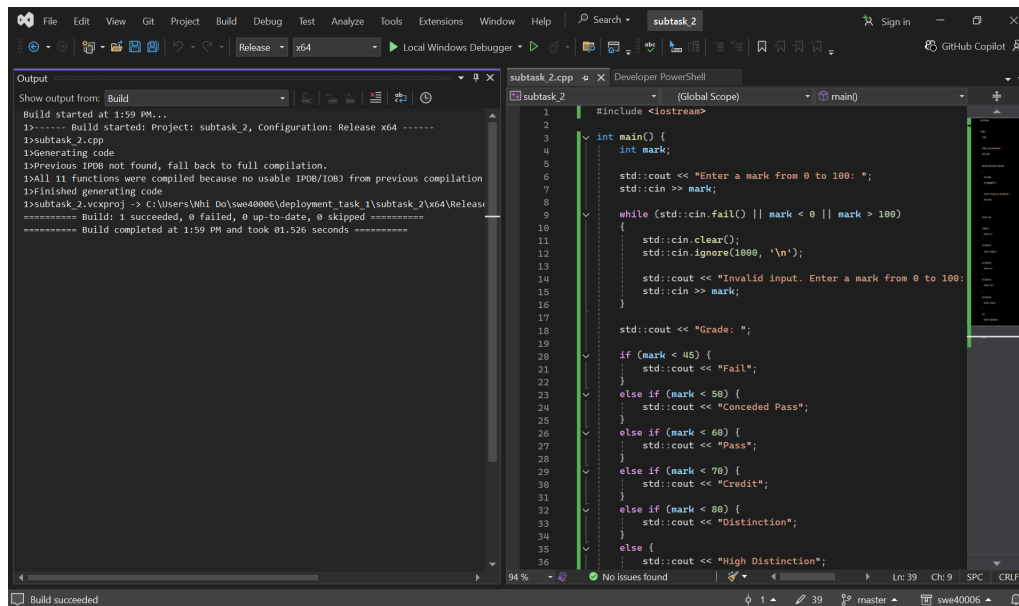


Figure 12: Building the project in Release mode.

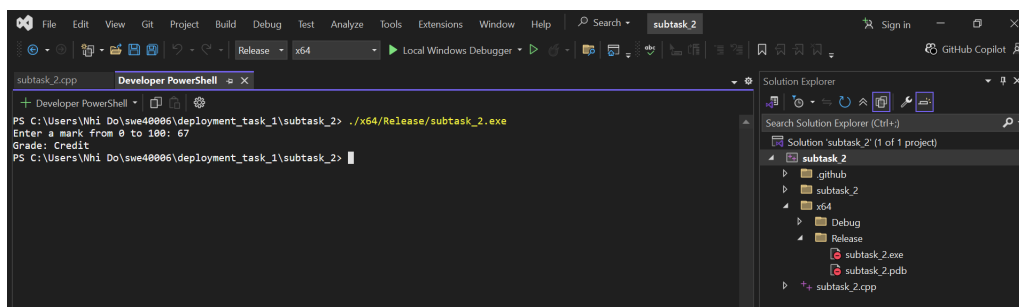


Figure 13: Verifying that the executable exists and runs correctly. The executable is created in the x64/Release directory as seen in the Solution Explorer.

Similarly, a WiX installer application is created, and a .wxs file is written to configure the installer.

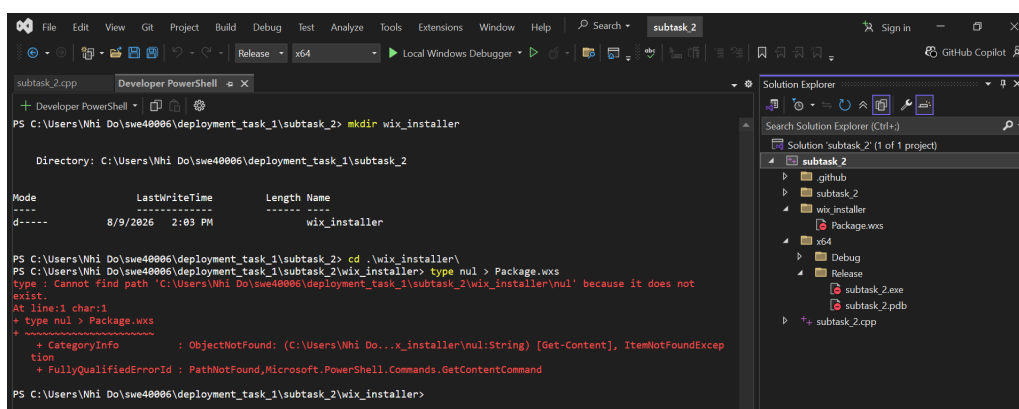


Figure 14: Creating the WiX installer project and source file.

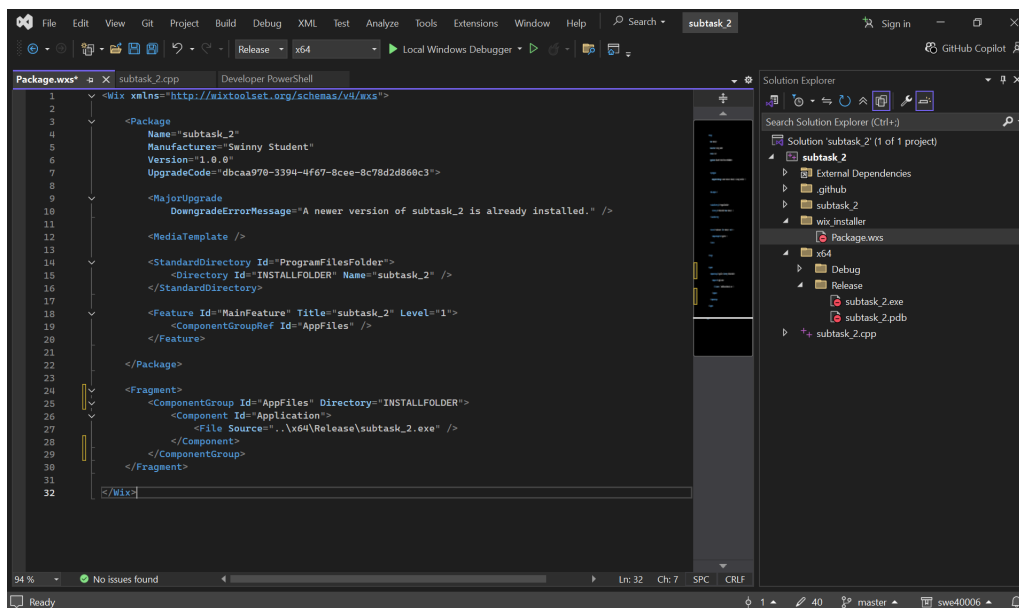


Figure 15: Writing the .wxs file.

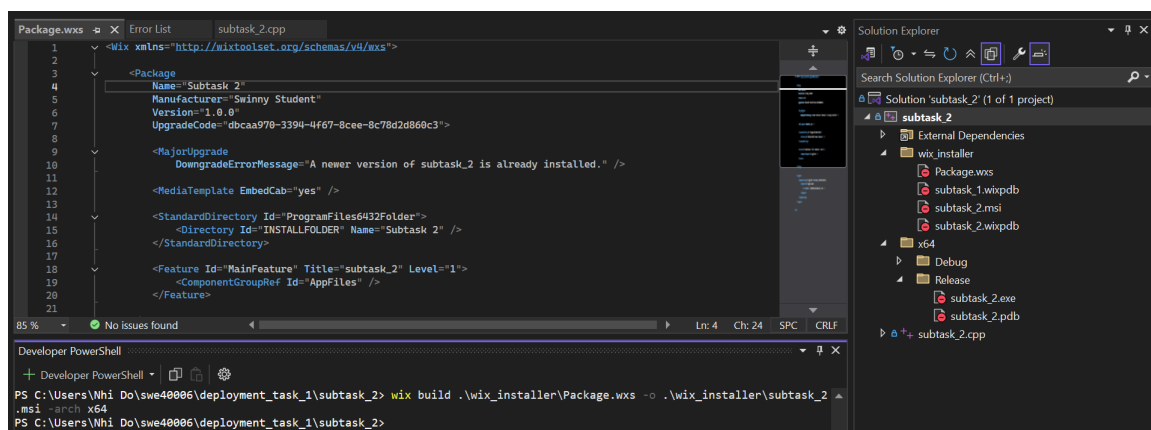


Figure 16: Building the MSI package.

Running the MSI package installs the app in the Program Files folder. The application runs as intended.

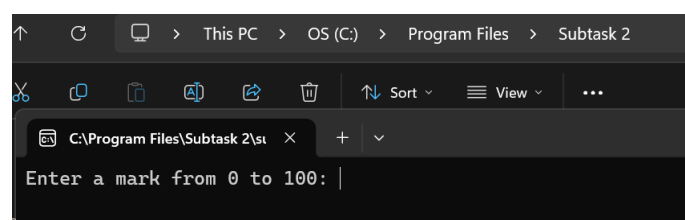


Figure 17: The application is installed in the Program Files folder. Double-clicking the app executes the program.

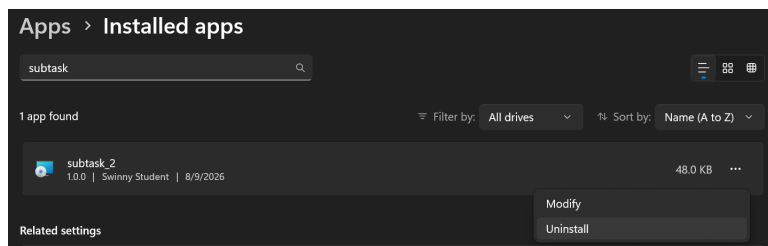


Figure 18: Uninstalling the application.

2.3 Task 1.3

The application created for Task 1.3 is a simple WinForms application, making the use of the packages `Newtonsoft.Json` and `Serilog` to demonstrate the packaging of multiple external DLLs.

First, I created the WinForms application using the template provided in Visual Studio.

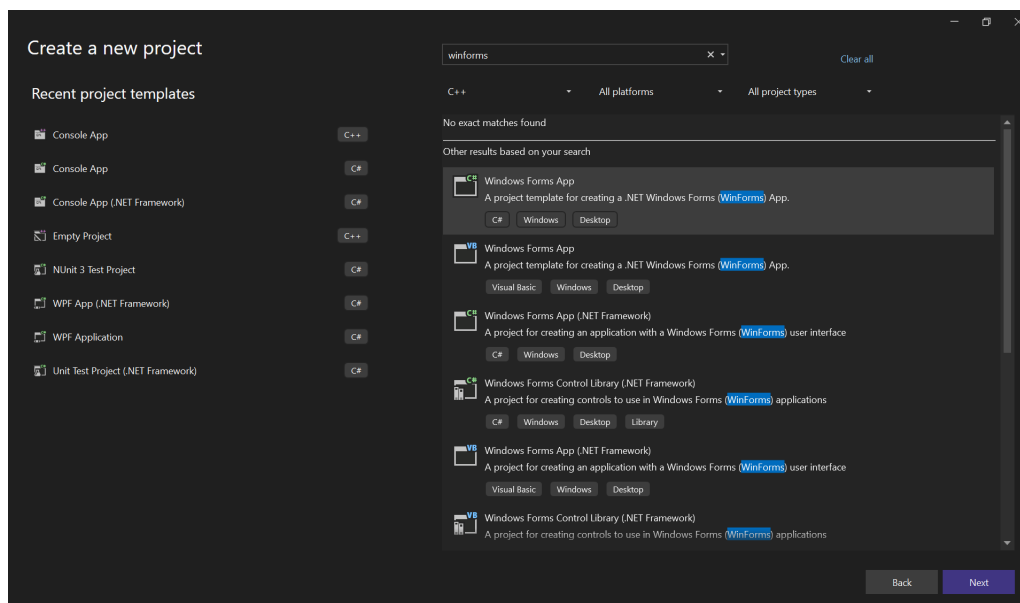


Figure 19: Creating a new WinForms project.

The application simply displays a window with a button which shows a messages and logs some information when it is clicked. The screenshot below shows that it compiles and runs successfully.

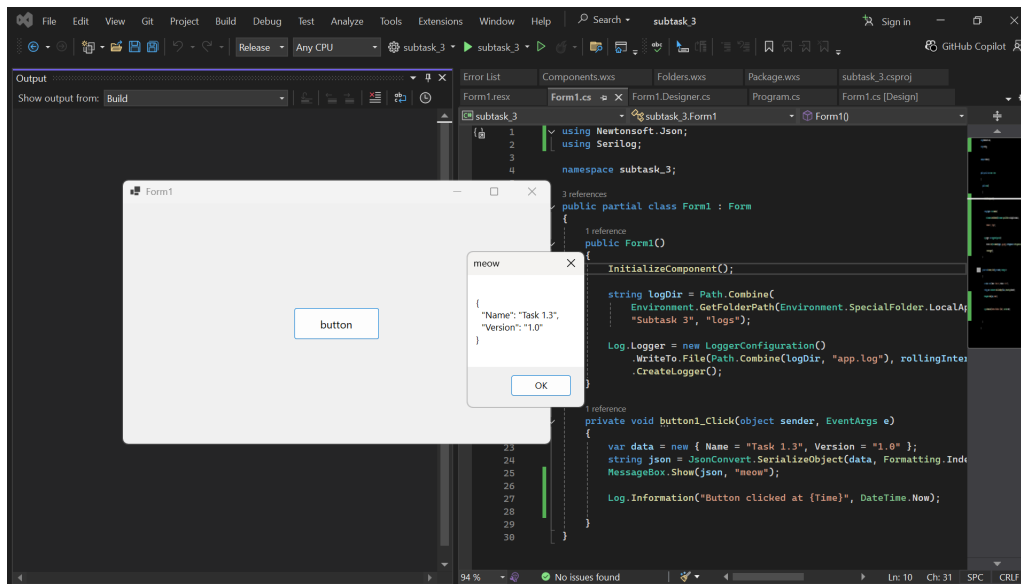


Figure 20: The app successfully runs.

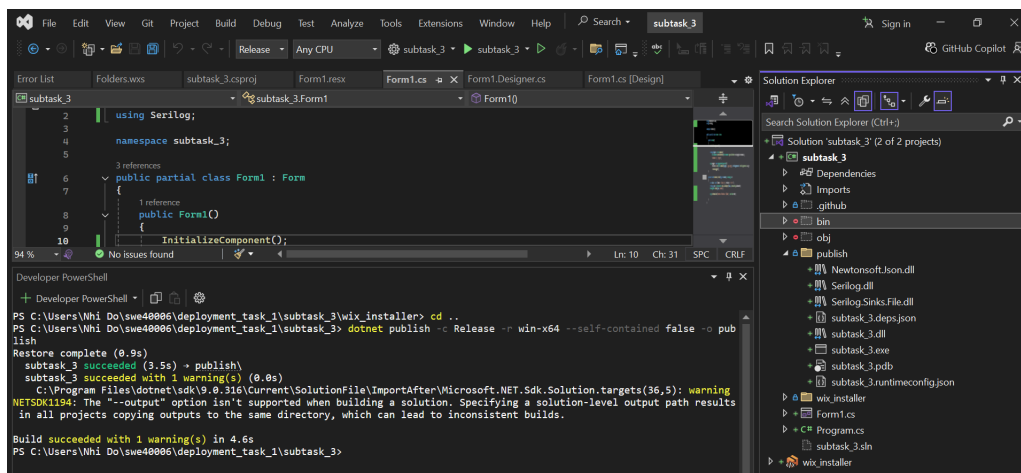


Figure 21: The project is published into a publish directory. The Solution Explorer on the right shows that the executable, runtime dependencies, and DLLs were created.

Next, I created a new WiX project, this time using the template provided by the HeatWave extension.

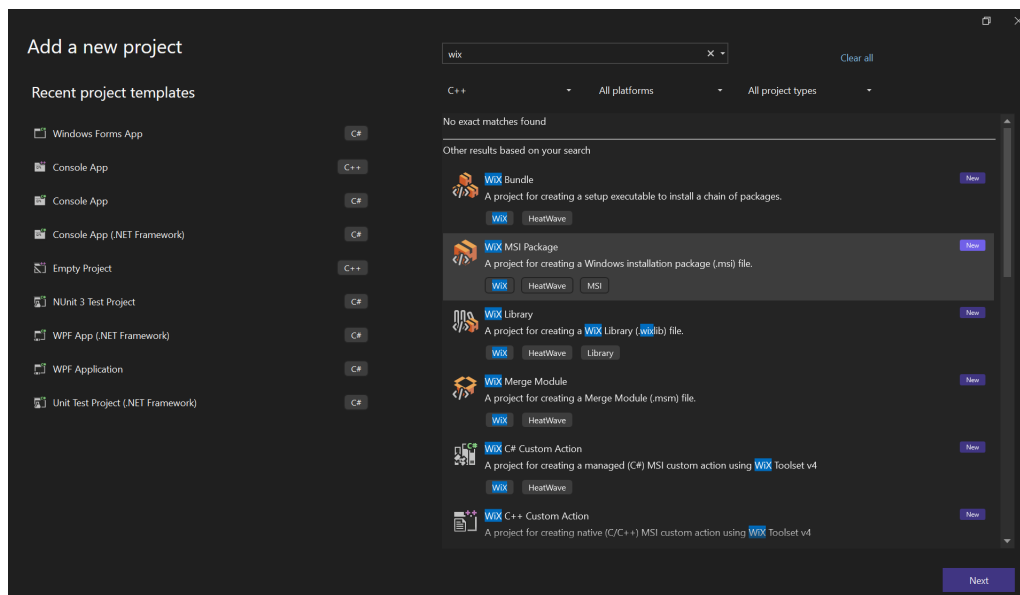


Figure 22: Creating a new WiX MSI Package project.

The **Components.wxs** file was configured to include the build outputs. This was done by authoring `<Component>` tags for each file, including the main executable, the DLL files, and others. Other build configurations were written in **Package.wxs** and **Folders.wxs**.

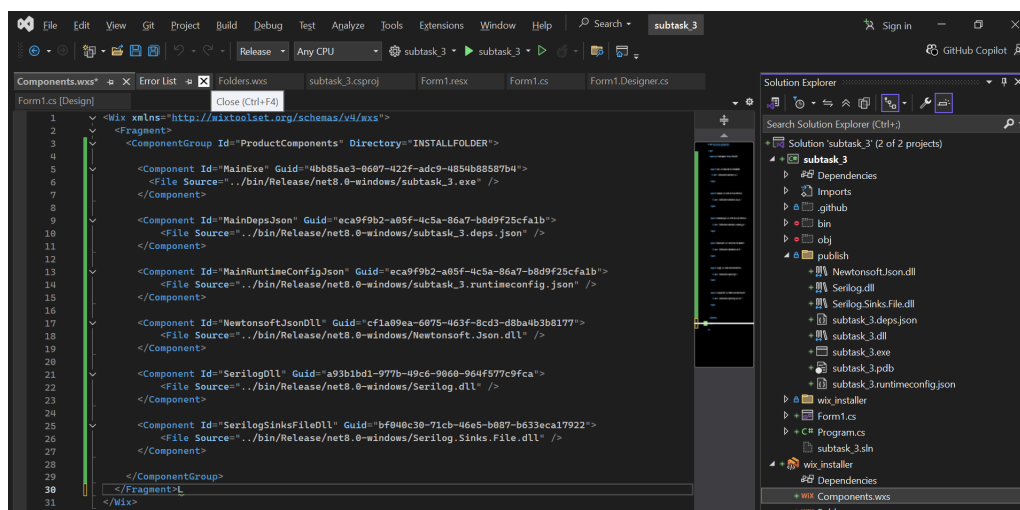


Figure 23: Writing the executable, DLLs, and dependencies into the script. These files would be included in the final MSI package.

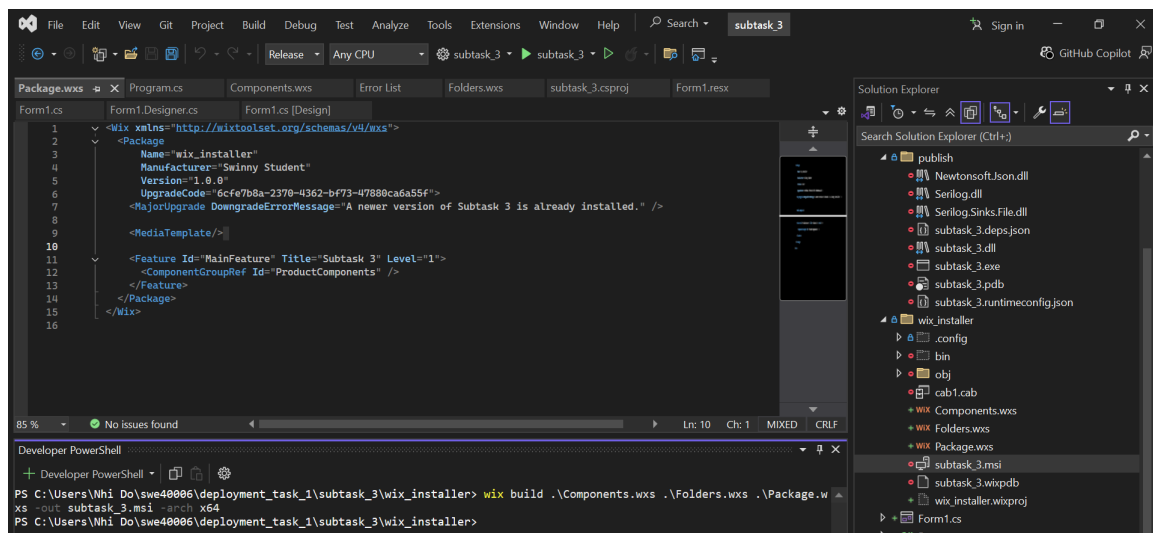


Figure 24: The WiX project is built to create an MSI package.

The application is installed in the Program Files folder. The screenshot below proves that the app was installed and executed without any errors.

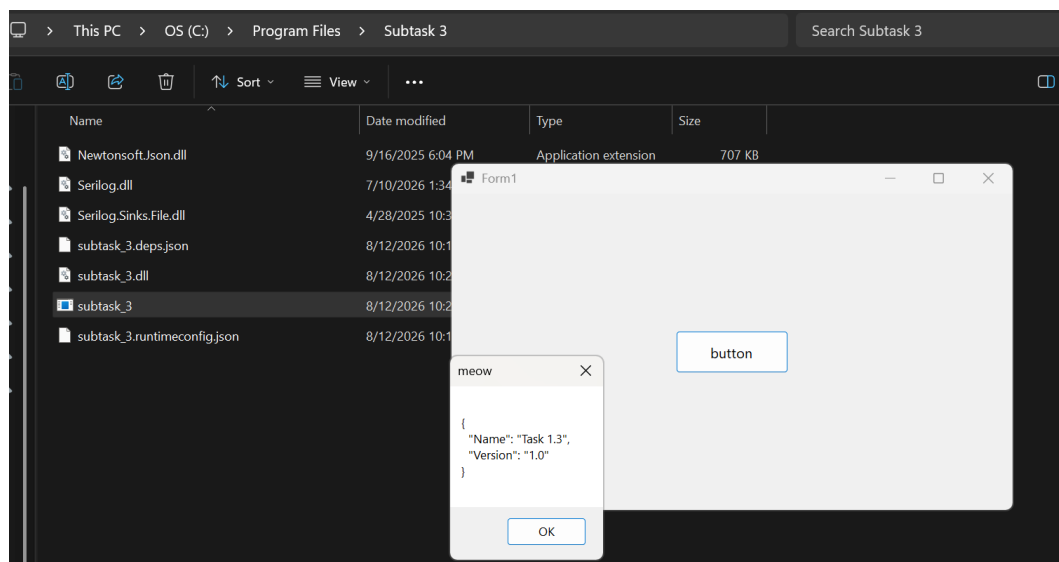


Figure 25: The application is installed using the MSI installer and runs successfully.

2.4 Task 1.4

For this task, I hosted the MSI package created in Task 1.1 on Github Releases and submitted a valid package manifest Pull Request (PR) to the `microsoft/winget-pkgs` repository.

First, I created a dedicated Github repository for this task and published the MSI package as a new Github Release. The Github repo can be found at <https://github.com/nhydny/Subtask-4>, and the direct link to the release is at <https://github.com/nhydny/Subtask-4/releases/tag/1.0.0>.

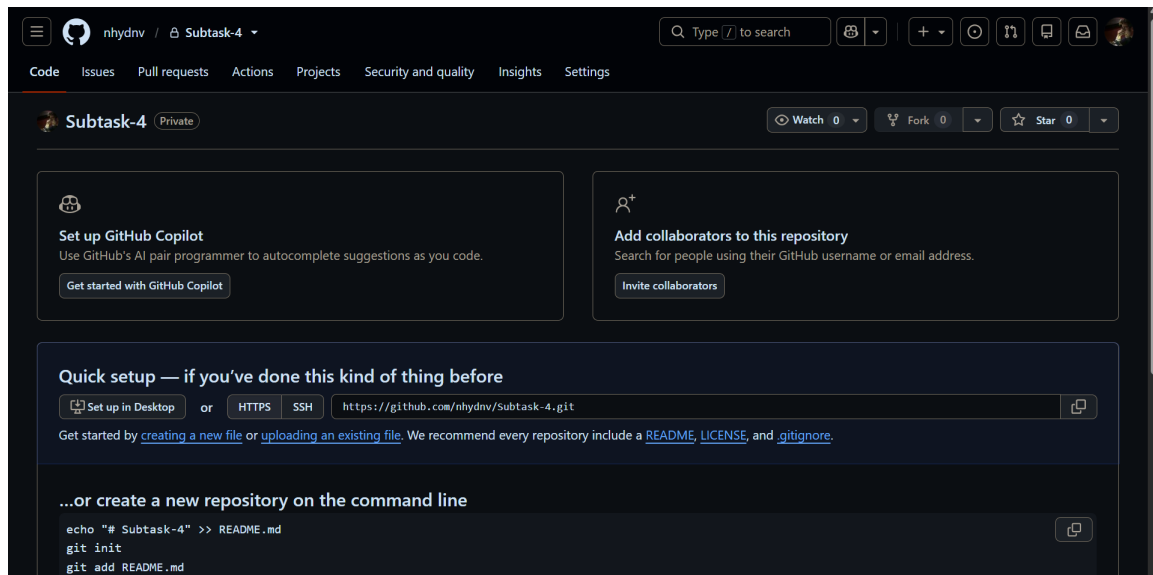


Figure 26: A dedicated Github repo is created to host the new release.

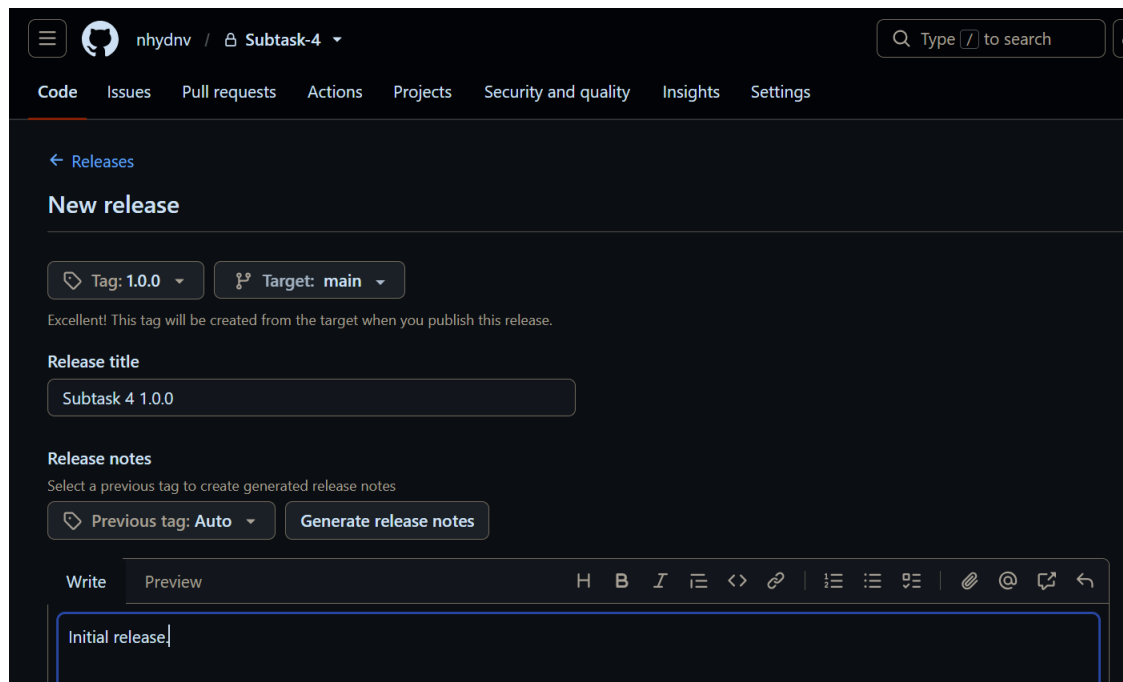


Figure 27: Creating a new Github Release and configuring its tag, title and other information. The MSI package is also uploaded.

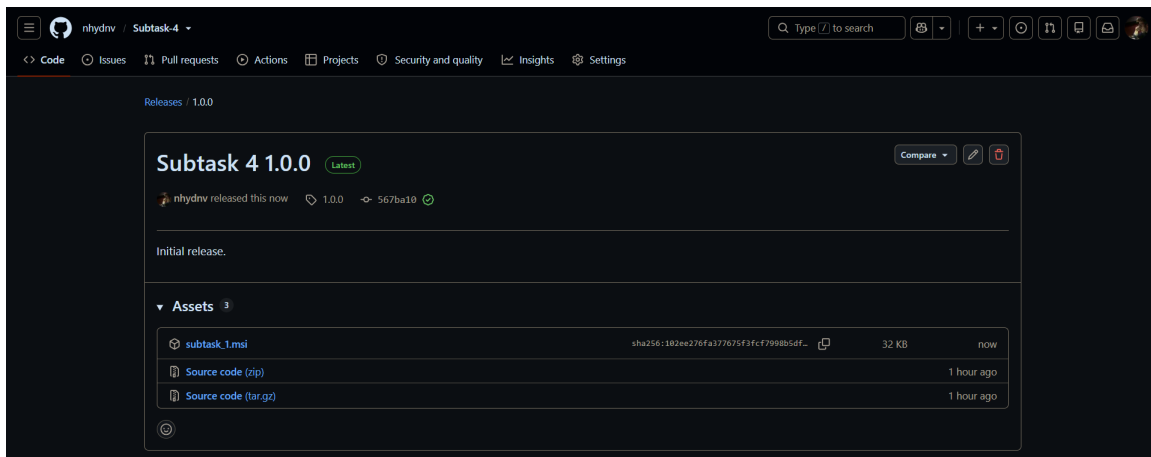


Figure 28: The release is published.

Once the MSI has been hosted on Github Releases, I tried submitting it to the Windows Package Manager. First, I installed WingetCreate, a tool to create, update, and submit manifest files to the WinGet repository.

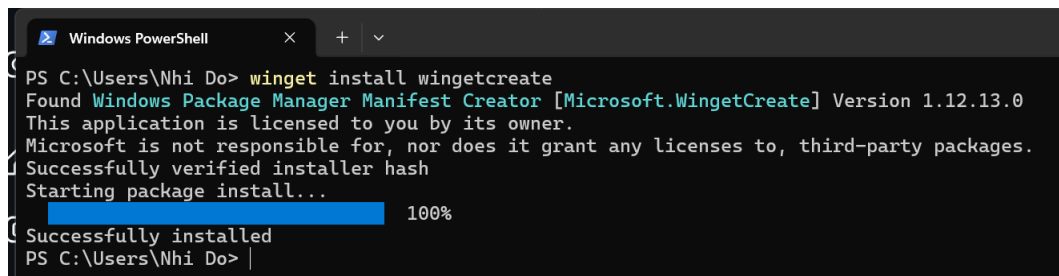


Figure 29: Installing WingetCreate.

Using WingetCreate, I created a new manifest, passing the URL to the installer hosted on Github as a command-line argument. This created a new manifest where I could specify each field by answering questions prompted by the tool.

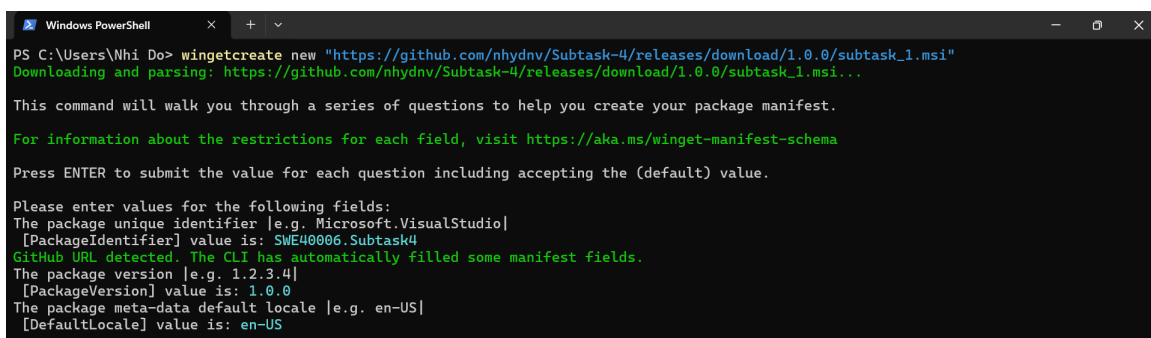


Figure 30: Creating a new manifest using WingetCreate.

I then authorised WingetCreate so that it could automatically create a fork of the `microsoft/winget-pkgs` repository with my account, as well as a new branch containing my manifest files. It also created a new PR to the pull the changes on my branch to the `microsoft/winget-pkgs:master` branch.

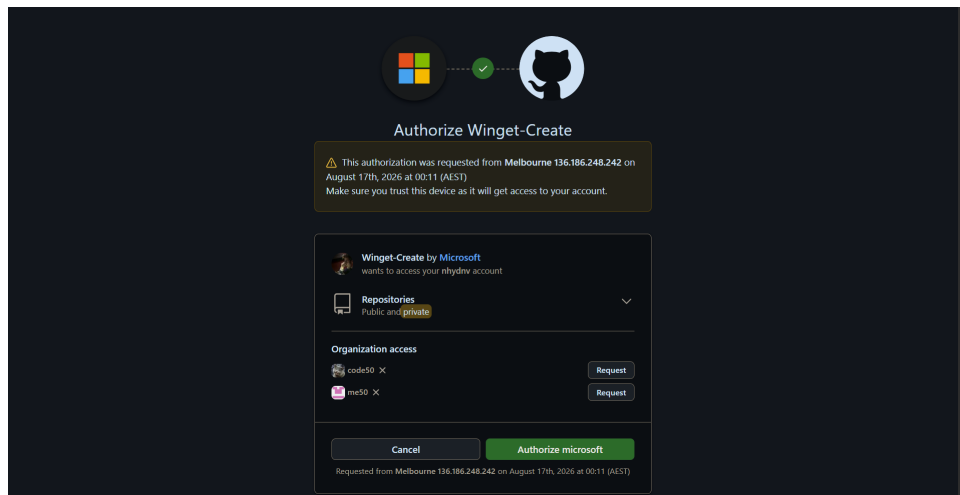
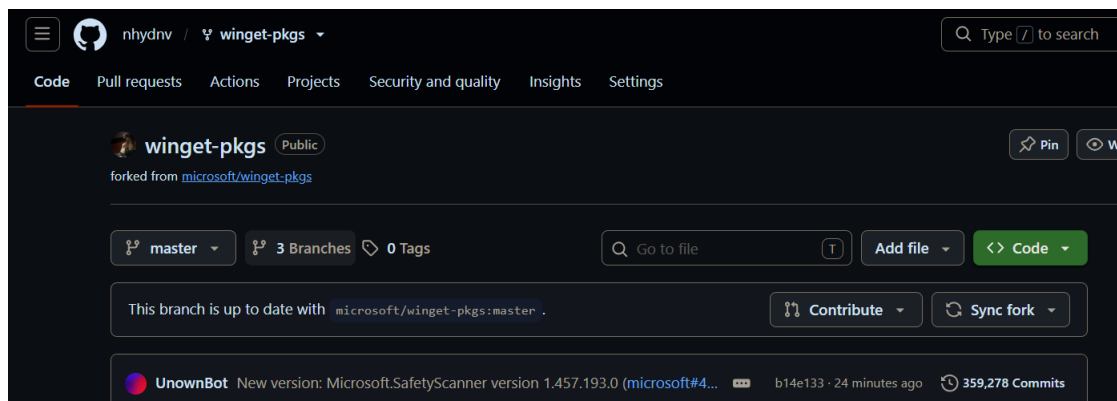


Figure 31: Authorising WingetCreate.

Figure 32: A fork of `microsoft/winget-pkgs` is created on my behalf by the WingetCreate tool.

Before the pull request could be resolved, the package manager needs to run tests against the installer and installed binaries to validate my submission. The screenshot below shows the checks in progress by the WinGet validator.

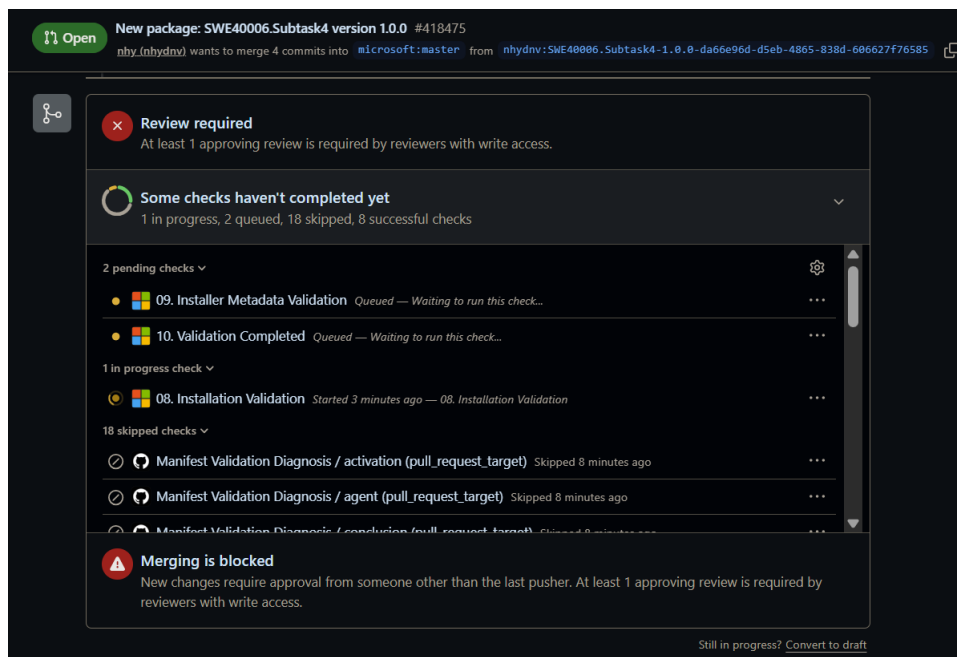


Figure 33: The WinGet validator checking the contents of my pull request before merging it to the master branch.

Originally, my WinGet submission was failing the installation validation check (**Validation-Shell-Execute**) because my WiX-built MSI referenced an external cabinet (.cab) file. WiX's default behaviour was to write the compressed app files into a separate **cab1.cab** file that has to sit next to the .msi file at install time, rather than embedding them inside the .msi itself. Since only the standalone MSI file was uploaded to Github Release, **msiexec** failed because it couldn't find the accompanying cabinet file containing the actual application files. This was resolved by adding `<MediaTemplate EmbedCab="yes"/>` to my .wxs script to create a self-contained .msi.

With this fix, all the automated validation checks for the pull request passed successfully; however, it still needs to be manually reviewed and approved by a community moderator before the package is available on the WinGet catalogue. In the meantime, a local installation using the exact same manifest submitted in the pull request was done to test verify that the package installs and functions correctly.

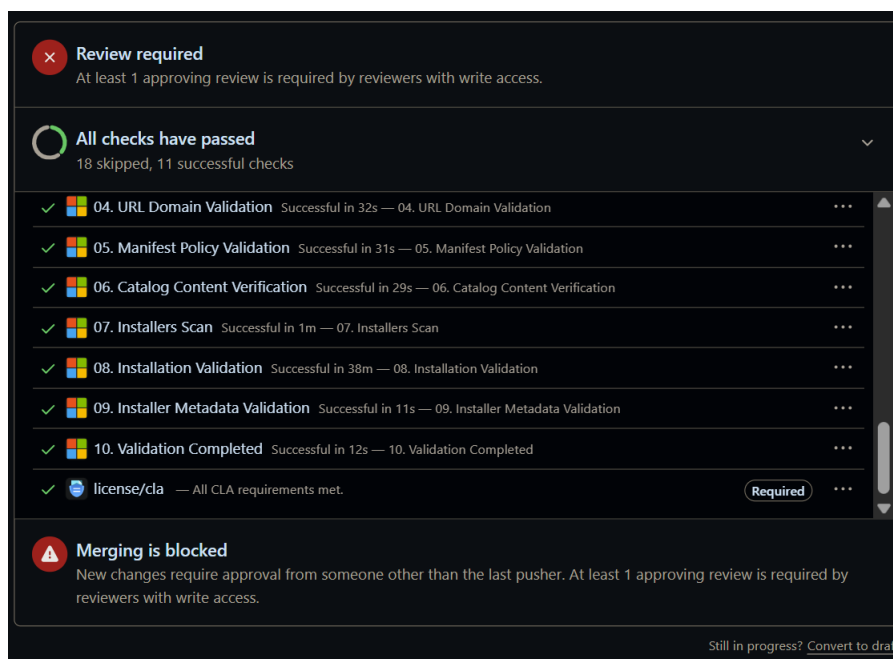


Figure 34: All validation checks were either skipped or passed.

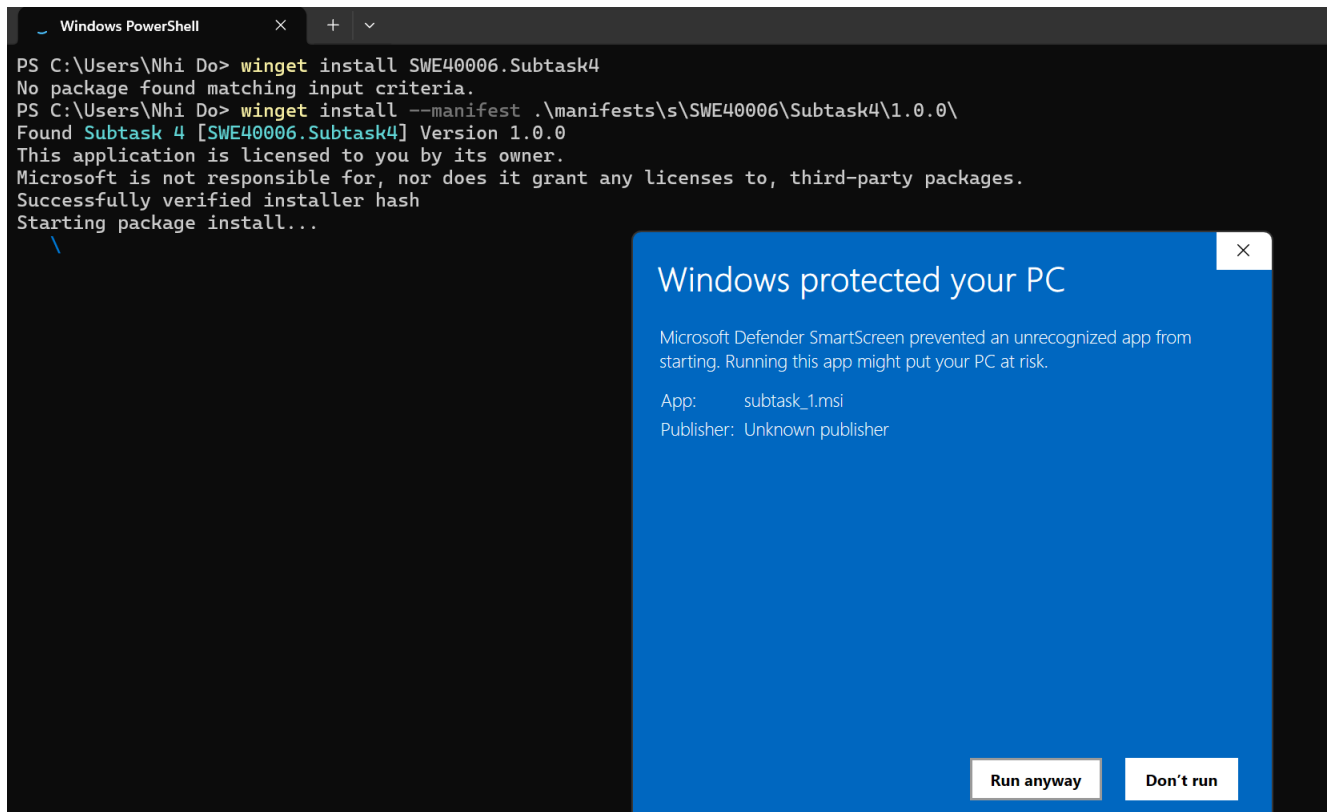


Figure 35: Local installation using the same manifest that was submitted.